

PLAN DÉTAILLÉ V2

Vector

Commando IA — Analyse de Données

★ avec 8 wow features ★

Durée	30 jours · 5 jours / semaine · 6 semaines
Pipelines	Data Analysis (A) + RAG textuel (B) — option B
Stack	FastAPI · Next.js · PostgreSQL+pgvector · OpenAI
Volume cible	100 000 lignes par dataset
Livrables	6 démos Loom hebdo + rapport pro final
Distinction	8 wow features pour un livrable impressionnant
Date	Juin 2026

Document de référence quotidien. Chaque jour est détaillé : objectif, tâches, outils, concepts à maîtriser, livrable attendu. Les wow features sont marquées ★.

Table des matières

1. Vision et différenciation du projet	3
2. Les 8 wow features qui font la différence	4
3. Vue globale des 6 semaines	6
4. Semaine 1 — Foundation	7
5. Semaine 2 — Ingestion des données	11
6. Semaine 3 — KPI configurables	15
7. Semaine 4 — Génération du dashboard HTML	19
8. Semaine 5 — Analyses spécialisées + RAG	23
9. Semaine 6 — Wow features, polish, rapport	27
10. Risques et plan de mitigation	31
11. Conseils pratiques au quotidien	32

1. Vision et différenciation du projet

Beaucoup de projets de stage construisent une « app qui marche ». Notre objectif est plus ambitieux : construire une application qui **impressionne**. Concrètement, l'encadrant doit se dire en regardant la démo finale : « Ce stagiaire ne s'est pas contenté du minimum, il a ajouté de la valeur partout ».

1.1 Ce qui rend un projet impressionnant

- **L'UX soignée** — animations fluides, états de chargement réfléchis, messages d'erreur clairs
- **Les détails techniques** — gestion des cas limites, robustesse aux erreurs LLM, performance
- **Les features qu'on n'attendait pas** — citations, comparaisons, raffinement conversationnel
- **La transparence du raisonnement** — Vector explique comment il pense, montre ses sources
- **Le polish visuel** — typographie, espacement, cohérence chromatique avec la plateforme
- **Les métriques mesurables** — temps de génération, coût LLM, taux de succès

1.2 Notre proposition de valeur unique

Vector ne sera pas juste « un outil qui fait des dashboards ». Vector sera :

- Un **partenaire d'analyse** qui combine données chiffrées (CSV) et contexte business (PDF)
- Un **analyste transparent** qui montre ses sources et explique son raisonnement
- Un **générateur intelligent** qui propose des KPI pertinents avec justification
- Un **outil de production** qui exporte en PDF prêt à présenter à la direction
- Une **interface conversationnelle** qui permet d'affiner sans repartir de zéro

2. Les 8 wow features qui font la différence

Ces 8 features sont disséminées dans le plan, intégrées au bon moment. Elles ne sont pas optionnelles — elles font partie du livrable. Marquées ★ dans les fiches jour.

#	Wow Feature	Semaine
1	Data Quality Scoring — Vector évalue la qualité du dataset sur 100 et signale les problèmes	S2 J3
2	KPI suggérés avec justification — chaque KPI proposé vient avec un « why »	S3 J4
3	Streaming du dashboard par section — le dashboard se construit visiblement section par section	S4 J2
4	Citation System — Vector indique quelles lignes du dataset et quels chunks de docs il a utilisés	S5 J2
5	Data Quality Warnings dans le dashboard — drapeaux visuels sur les KPI peu fiables	S5 J4
6	Mode Comparaison — comparer deux périodes côte à côte dans un même dashboard	S6 J1
7	Conversational Refinement — « Refais le dashboard mais exclus la région Nord »	S6 J1
8	Smart Caching — embeddings et profils mis en cache, pas de recalcul inutile	S6 J2

2.1 Pourquoi ces 8-là

- **Citation system** : élément #1 de confiance dans les LLM en entreprise. C'est l'arme anti-hallucination, et un sujet brûlant en 2026.
- **Data Quality Scoring** : montre que Vector ne fait pas semblant. Il ouvre la donnée, l'analyse, en pointe les faiblesses avant de tirer des conclusions.
- **KPI justifiés** : transforme une simple liste en conseil métier. L'utilisateur comprend pourquoi.
- **Streaming par section** : effet « waouh » immédiat à la démo. L'utilisateur voit le dashboard se construire en direct.
- **Mode Comparaison** : besoin business universel. C'est la première question que pose un commercial.
- **Conversational Refinement** : démontre la puissance du LLM au-delà du « bouton qui exécute ».
- **Smart Caching** : montre la maturité technique. On ne paie pas OpenAI deux fois pour la même chose.

✓ L'effet cumulé

Chacune de ces features prise séparément est jolie. **Les 8 ensemble** donnent un produit qui ressemble à du logiciel d'éditeur sérieux, pas à un projet de stage. C'est ce qui fait basculer la perception de l'encadrant.

3. Vue globale des 6 semaines

Se mai ne	Thème	Démo cible	Wow features
S1	Foundation	Vector existe, répond en streaming	—
S2	Ingestion des données	Upload CSV → analyse textuelle structurée	★1 Data Quality Scoring
S3	KPI configurables	KPI suggérés, édités, recalculés	★2 KPI avec justification
S4	Dashboard HTML	Dashboard pro avec KPI cards + recommandations	★3 Streaming par section
S5	Analyses spécialisées + RAG	5 actions opérationnelles + PDF + RAG documents	★4 Citations, ★5 Quality Warnings
S6	Wow features + polish + rapport	Démonstration finale scénarisée	★6 Comparaison, ★7 Refinement, ★8 Caching

3.1 La progression logique

Chaque semaine construit sur la précédente. Pas de saut, pas de retour en arrière :

- **S1** pose les fondations (auth, DB, streaming). Sans ça, rien ne tient.
- **S2** apporte les données. On peut maintenant analyser quelque chose.
- **S3** rend l'analyse configurable (KPI). L'utilisateur prend la main.
- **S4** matérialise l'analyse (dashboard HTML). Le livrable devient visible.
- **S5** multiplie les capacités (5 actions + RAG). Vector devient riche fonctionnellement.
- **S6** distille le polish et les features qui marquent. Le projet devient impressionnant.

✓ La règle d'or

À la fin de **chaque jour**, quelque chose de visible doit fonctionner. Pas de jour « invisible » où on code 8h sans rien à montrer. Ça maintient le moral, ça permet la démo hebdo, ça réduit le risque de découvrir un problème en fin de sprint.

Semaine 1 — Foundation

Objectif de la semaine

Mettre en place toute la fondation technique : auth, base de données, intégration OpenAI, streaming WebSocket, première version visible de Vector. À la fin de la semaine, l'utilisateur peut se connecter, voir Vector dans l'interface, lui dire bonjour, et recevoir une réponse en streaming. Aucune fonctionnalité métier (pas encore de dataset, pas de dashboard) — uniquement la tuyauterie qui doit être solide.

Démo vidéo de fin de semaine

Démo 1 : la stack tourne en local, je crée un compte, je me connecte, je vois Vector dans la liste des Commandos, je lui envoie « Présente-toi », il me répond en streaming.

S1 J1	Structure du projet et environnement Docker
Objectif	Avoir un squelette de projet complet (backend Python + frontend Next.js) avec un docker-compose qui démarre Postgres+pgvector et Redis. Aucun code applicatif encore, juste la structure et les dépendances.
Tâches	- Créer la structure de dossiers : backend/, frontend/, infra/, docs/ - Initialiser le projet Python dans backend/ avec un pyproject.toml et un environnement virtuel (Python 3.11) - Lister les dépendances Python : FastAPI, SQLAlchemy, asyncpg, Alembic, Pydantic, python-jose, bcrypt, openai, pandas, openpyxl, pypdf, python-docx, tiktoken, celery, redis, jinja2, pgvector - Initialiser le projet Next.js dans frontend/ avec pnpm : Tailwind v4, Radix, TanStack Query, Zustand, ApexCharts, lucide-react - Écrire docker-compose.yml avec 3 services : db (pgvector/pgvector:pg16), redis (redis:7), backend (Dockerfile à venir) - Créer .env.example pour backend et frontend .gitignore propre (exclure .env, node_modules, __pycache__, .venv) - Premier git commit : « feat: initial project structure »
Outils	Docker, Docker Compose, pnpm, py -3.11, git, VS Code
À maîtriser	Structure monorepo (frontend et backend dans le même repo), variables d'environnement (.env vs .env.example), Docker Compose orchestration
Livrable	Arborescence complète. docker compose up db redis démarre Postgres+pgvector et Redis sans erreur.

S1 J2	Conception de la base de données et migrations
Objectif	Définir le schéma complet des 12 tables et créer la première migration Alembic. Active l'extension pgvector. Migration testée en upgrade et downgrade.
Tâches	- Dessiner le schéma complet sur draw.io ou Excalidraw (12 tables, relations, clés étrangères) - Configurer SQLAlchemy 2.0 async dans backend/db/session.py avec connection pool - Configurer Alembic dans backend/db/migrations/ et son alembic.ini - Créer les modèles SQLAlchemy dans backend/db/models/ : User, Agent, Conversation, Message, Dataset, DatasetColumn, KPI, KPIValue, Dashboard, Analysis, Document, Chunk - Première migration : CREATE EXTENSION pgvector + toutes les tables - La colonne chunks.embedding est de type Vector(1536) - Index IVFFlat sur chunks.embedding avec vector_cosine_ops - Lancer alembic upgrade head, vérifier dans pgAdmin, lancer alembic downgrade base, vérifier que c'est vide - Pré-populer la table agents avec une entrée Vector
Outils	SQLAlchemy 2.0, Alembic, pgvector, DBeaver ou pgAdmin
À maîtriser	Schéma relationnel, migrations versionnées et réversibles, extension Postgres, index vectoriel IVFFlat
Livrable	12 tables visibles dans pgAdmin. Migration upgrade/downgrade fonctionne. Vector est dans la table agents.

S1 J3	Authentification JWT et premier endpoint protégé
Objectif	Permettre à un utilisateur de s'inscrire, de se connecter, et de récupérer la liste des agents (Vector) avec son token JWT.
Tâches	• Modèle User finalisé avec colonnes email, password_hash, created_at • Schémas Pydantic : UserCreate, UserLogin, UserResponse, TokenResponse • Module backend/auth/security.py : hash_password, verify_password (bcrypt), create_access_token, decode_token (python-jose) • Module backend/auth/dependencies.py : get_current_user (vérifie le JWT du header Authorization) • Endpoints POST /auth/register et POST /auth/login • Endpoint GET /agents protégé qui retourne la liste des agents • Endpoint GET /me qui retourne l'utilisateur courant • Tests unitaires pytest pour chaque endpoint • Documentation Swagger UI vérifiée à http://localhost:8000/docs
Outils	FastAPI, Pydantic v2, python-jose, bcrypt/passlib, pytest, httpx (tests)
À maîtriser	JWT (header/payload/signature), bcrypt hashing, FastAPI dependencies, validation Pydantic, tests unitaires
Livrable	Je peux register + login via Swagger et obtenir un JWT. Avec ce JWT, je peux appeler GET /agents.

S1 J4	Intégration OpenAI avec provider abstraction
Objectif	Construire une couche abstraite pour les appels LLM, pour pouvoir changer de provider plus tard sans réécrire tout. Tester le streaming et les structured outputs.
Tâches	• Interface abstraite backend/ai/providers/base.py avec méthodes : chat, chat_stream, embed, structured_output • Implémentation backend/ai/providers/openai.py avec le SDK officiel openai==1.54+ • Configuration des modèles dans backend/config.py : MODEL_CHAT = gpt-4o, MODEL_MINI = gpt-4o-mini, MODEL_EMBED = text-embedding-3-small • Gestion d'erreurs : RateLimitError, APITimeoutError, APIConnectionError avec retry exponentiel • Tests d'intégration (avec mock pour ne pas brûler des tokens en CI) • Script de test manuel : envoyer « Bonjour, présente-toi » à GPT-4o et imprimer la réponse en streaming • Vérifier que les structured outputs marchent : prompt « Donne 3 fruits en JSON {fruits: [...]} » → JSON valide
Outils	openai SDK 1.54+, asyncio, tenacity (retry), pytest, pytest-asyncio
À maîtriser	Abstraction d'interface, async streaming, retry pattern, response_format JSON schema, mocking en tests
Livrable	Script scripts/test_openai.py qui envoie un message et reçoit la réponse en streaming. Tests passent.

S1 J5	Frontend foundation, chat WebSocket, première démo
Objectif	Construire le shell de l'application (sidebar + header + page Vector) et faire fonctionner un premier chat avec streaming WebSocket de bout en bout.
Tâches	• Configuration Tailwind v4 avec les variables de couleur de la plateforme (palette dark + accent vert Vector) • Layout principal dans app/layout.tsx : sidebar gauche + header + main content • Page app/login/page.tsx et app/register/page.tsx • Store Zustand pour l'auth (token, current user) • Page app/vector/page.tsx avec : avatar Vector, titre, 5 cartes d'actions rapides (vides pour l'instant), input de chat • Endpoint backend WebSocket /ws/chat : reçoit un message, appelle Vector, stream la réponse token par token • Hook frontend useChat qui ouvre le WebSocket et gère les messages • Rendu Markdown des messages avec react-markdown • Tester avec « Présente-toi en 3 phrases » • Enregistrer la Démo 1 Loom (3-4 minutes)
Outils	Next.js 16, Tailwind v4, Radix UI, Zustand, WebSocket API (navigateur), react-markdown, Loom
À maîtriser	App Router de Next.js, layouts imbriqués, hooks React, WebSocket dans le navigateur, streaming d'UI
Livrable	Démo 1 enregistrée et envoyée à l'encadrant. Vector existe visuellement, répond en streaming.

Semaine 2 — Ingestion des données

Objectif de la semaine

Permettre à l'utilisateur d'uploader des fichiers CSV ou Excel, de les parser, d'en extraire un profil statistique riche, et d'obtenir une première analyse textuelle structurée de Vector. C'est la fondation du Pipeline A (Data Analysis). Première wow feature : Data Quality Scoring.

Démo vidéo de fin de semaine

Démo 2 : j'uploade un CSV de ventes, le système détecte ses colonnes, calcule un profil et un score de qualité, je clique « Analyse », Vector me produit une analyse texte structurée en 4 sections (Vue d'ensemble, Tendances, Anomalies, Recommandations).

S2 J1	Modèle datasets et API d'upload
Objectif	Permettre l'upload de fichiers CSV/XLSX, leur stockage, et leur listing. Pas encore de parsing — juste la mécanique d'upload.
Tâches	- Finaliser les modèles SQLAlchemy Dataset (id, user_id, name, file_path, file_size, row_count, column_count, status, created_at) et DatasetColumn - Schémas Pydantic : DatasetCreate, DatasetResponse, DatasetListItem - Endpoint POST /datasets avec FastAPI UploadFile : reçoit multipart, valide MIME et taille (max 100 Mo), stocke dans uploads/datasets/{user_id}/{uuid}.csv - Endpoint GET /datasets : liste les datasets de l'utilisateur courant - Endpoint DELETE /datasets/{id} avec suppression du fichier sur disque - Statut « uploaded » dans la base à ce stade (le parsing viendra J2) - Côté frontend : composant DropZone avec react-dropzone, barre de progression d'upload - Page app/datasets/page.tsx qui liste les datasets uploadés
Outils	FastAPI UploadFile, aiofiles, react-dropzone, TanStack Query
À maîtriser	Upload multipart/form-data, validation MIME, stockage de fichiers, sécurité (filtrer les types)
Livrable	Je peux uploader un CSV depuis l'UI, je le vois apparaître dans la liste, je peux le supprimer.

S2 J2	Parsing pandas et détection automatique des types
Objectif	Quand l'utilisateur upload un dataset, on parse le fichier, on détecte les types de chaque colonne, on sauvegarde les métadonnées.
Tâches	• Module backend/datasets/parser.py avec fonctions parse_csv(path) et parse_xlsx(path) • Détection automatique de l'encodage CSV avec chardet (utf-8, latin-1, etc.) • Détection des séparateurs CSV (virgule, point-virgule, tab) • Pour chaque colonne : type pandas (int64, float64, object, datetime64), nullable, unique_count, sample_values (5 valeurs représentatives) • Tentative de conversion en datetime si la colonne semble être une date • Sauvegarde dans dataset_columns avec tous ces métadonnées • Mise à jour du statut Dataset : « parsing » → « ready » • Si parsing échoue : statut « error » avec message • Endpoint GET /datasets/{id}/schema qui retourne la liste des colonnes • Côté frontend : page détail du dataset avec table des colonnes et leurs types
Outils	pandas, chardet, openpyxl, dateutil
À maîtriser	Détection d'encodage, dtypes pandas, conversion de types, gestion d'erreurs de parsing
Livrable	À l'upload, le dataset est parsé automatiquement, ses colonnes apparaissent dans l'UI avec leurs types.

S2 J3	Profilage statistique avancé + Data Quality Scoring
Objectif	Calculer un profil statistique riche de chaque dataset. Ajouter notre première wow feature : un score de qualité des données sur 100.
Tâches	• Module backend/datasets/profiler.py avec fonction profile_dataset(dataset_id) -> dict • Statistiques par colonne numérique : min, max, mean, median, std, percentiles (25, 50, 75) • Statistiques par colonne catégorielle : top-20 valeurs + count, cardinalité totale • Si > 50 valeurs uniques, on regroupe : top-20 + « Autres » avec leur somme • Détection de la colonne date principale (heuristique : nom contient date/time/created/jour ET dtype datetime) • Calcul des corrélations principales entre colonnes numériques (Pearson) • ★ Data Quality Scoring : algo qui note le dataset sur 100 selon : taux de valeurs manquantes (-X par colonne touchée), doublons (-X), incohérences de type, valeurs aberrantes (>3 sigma) • Liste des « issues » détectées : « 18% de valeurs manquantes sur la colonne email », « 142 doublons exacts détectés », etc. • Endpoint GET /datasets/{id}/profile qui retourne tout le profil + score + issues • UI : sur la page dataset, afficher le badge « Score : 87/100 » avec couleur (vert >80, ambre 60-80, rouge <60), et la liste des issues
Outils	pandas, numpy, scipy
À maîtriser	Statistiques descriptives, détection d'outliers (z-score, IQR), qualité de données, agrégation
Livrable	Chaque dataset affiche son score qualité et la liste de ses problèmes détectés.
★ WOW	Data Quality Scoring — l'utilisateur voit immédiatement si son dataset est exploitable sans devoir l'inspecter manuellement.

S2 J4	Premier prompt d'analyse Vector
Objectif	Vector produit sa première analyse textuelle structurée à partir du profil d'un dataset.
Tâches	• Module backend/vector/analyzer.py • Prompt système Vector long et précis : rôle, ton, structure de réponse en 4 sections (Vue d'ensemble, Tendances, Anomalies, Recommandations), instruction de citer les colonnes mentionnées • Fonction async analyze_dataset(dataset_id, question) -> AsyncIterator[str] qui yield les tokens • Le prompt utilisateur inclut le profil JSON pré-calculé (pas les données brutes) • Endpoint WebSocket /ws/datasets/{id}/analyze qui stream la réponse • Côté frontend : page dataset avec bouton « Demander une analyse » qui ouvre une zone de chat • Rendu Markdown en streaming dans la zone de chat • Sauvegarde de l'analyse dans la table analyses pour pouvoir la relire
Outils	OpenAI streaming, WebSocket, react-markdown
À maîtriser	Prompt engineering avancé, structure de réponse, streaming en pratique, sauvegarde de l'historique
Livrable	Sur n'importe quel dataset uploadé, je peux demander une analyse et recevoir 4 sections en streaming Markdown.

S2 J5	Conversation memory + Démo 2
Objectif	Quand l'utilisateur reprend une conversation, Vector se souvient du contexte. Tests sur 3 datasets variés pour valider la robustesse. Enregistrement de la démo 2.
Tâches	• Sauvegarder chaque message (user + assistant) dans la table messages avec un conversation_id • Endpoint POST /conversations pour créer une nouvelle conversation • Endpoint GET /conversations pour lister celles de l'utilisateur • Quand on reprend une conversation existante, charger les N derniers messages et les injecter dans le prompt comme historique • Sidebar gauche : panel central avec liste des discussions passées (style des autres agents de la plateforme) • Test sur 3 datasets différents : ventes 2024 (10k lignes), campagne marketing (5k lignes), RH (1k lignes) • Ajustements du prompt selon les résultats • Enregistrer la Démo 2 Loom (3-5 minutes)
Outils	SQLAlchemy, TanStack Query, OpenAI
À maîtriser	Mémoire conversationnelle, fenêtre de contexte, troncature intelligente de l'historique
Livrable	Démo 2 enregistrée. Chat persistant, contextuel, fonctionne sur 3 types de datasets.

Semaine 3 — KPI configurables

Objectif de la semaine

L'utilisateur peut créer, éditer, supprimer des KPI. Vector en suggère automatiquement à partir du profil du dataset. Chaque KPI suggéré vient avec sa justification (wow feature #2). Sécurité : les formules KPI sont parsées via une whitelist AST, pas d'eval libre.

Démo vidéo de fin de semaine

Démo 3 : j'uploade un dataset, je clique « Suggérer KPI », Vector me propose 8 KPI avec leur raison d'être, j'en accepte 5, j'en modifie 2 formules, j'en ajoute 1 manuellement, je vois les valeurs calculées en temps réel.

S3 J1	Modèle KPI et parser de formules sécurisé
Objectif	Concevoir la table des KPI et construire un parser de formules basé sur l'AST Python avec whitelist stricte des fonctions et opérateurs autorisés.
Tâches	- Finaliser les modèles KPI (id, dataset_id, name, description, formula, unit, target_value, format, display_order) et KPIValue (id, kpi_id, value, computed_at) - Schémas Pydantic associés - Module backend/kpis/formula_parser.py - Parser AST qui valide une formule : seuls les noms autorisés (fonctions whitelisted) et opérateurs (+, -, *, /, %, parenthèses) sont permis - Whitelist fonctions : sum, mean, count, distinct, min, max, median, percentile, std, var, abs - Toute autre construction (import, exec, open, eval, lambda, list comprehension custom) est rejetée à l'analyse - Fonction execute_formula(formula, df) -> float qui évalue la formule sur un DataFrame pandas - Tests de sécurité : __import__('os'), open('/etc/passwd'), exec('...') doivent tous échouer - Tests fonctionnels : sum(montant), mean(montant), distinct(client_id), sum(montant) / count(distinct(client_id))
Outils	ast (stdlib Python), pandas, pytest
À maîtriser	AST Python, whitelisting, sécurité des évaluations de chaînes, sandboxing
Livrable	Parser de formules testé : sécurisé contre les injections, fonctionne sur les formules métier classiques.

S3 J2	API KPI CRUD complet
Objectif	Endpoints REST complets pour gérer les KPI : créer, lister, modifier, supprimer, recalculer.
Tâches	• GET /datasets/{id}/kpis : liste des KPI d'un dataset, ordonnés par display_order • POST /datasets/{id}/kpis : créer un KPI, validation de la formule, recalcul immédiat • PATCH /kpis/{id} : modifier (nom, formule, etc.), recalcul si formule changée • DELETE /kpis/{id} : suppression • POST /kpis/{id}/compute : forcer le recalcul (utile si dataset modifié) • POST /kpis/reorder : changer display_order de plusieurs KPI en une fois (drag-drop côté UI) • Tests unitaires de chaque endpoint, avec cas limites : formule invalide, colonne supprimée du dataset, formule qui retourne NaN • Documentation Swagger vérifiée
Outils	FastAPI, Pydantic, pandas, pytest
À maîtriser	REST CRUD complet, recalcul à la demande, gestion d'erreurs avec messages explicites pour l'UI
Livrable	Tous les endpoints KPI fonctionnent et sont testés. Swagger UI montre les schémas.

S3 J3	UI configuration KPI
Objectif	Interface complète pour gérer les KPI : tableau, ajout, édition avec validation en temps réel, drag-drop pour réordonner.
Tâches	• Page <code>app/datasets/[id]/kpis/page.tsx</code> • TanStack Table avec colonnes : Nom, Formule, Valeur actuelle, Cible, Format, Actions • Bouton « Ajouter KPI » qui ouvre une modale Radix Dialog • Modale d'édition avec champs : nom, description, formule, unité, target, format d'affichage • Champ Formule avec autocomplete des noms de colonnes du dataset (combobox) • Preview live de la valeur : pendant qu'on tape la formule, on appelle l'endpoint <code>compute</code> en <code>debounce 500ms</code> pour afficher la valeur résultante • Affichage d'erreur en temps réel si la formule est invalide • Drag-drop pour réordonner les KPI avec <code>dnd-kit</code> ou similaire • Confirmation avant suppression (alerte Radix)
Outils	Next.js, TanStack Table, TanStack Query, Radix Dialog, Radix AlertDialog, <code>dnd-kit</code>
À maîtriser	Mutations optimistes, <code>debounce</code> , drag-drop accessible, validation côté client + serveur
Livrable	L'utilisateur gère ses KPI entièrement depuis l'UI : créer, modifier, recalculer, réordonner, supprimer.

S3 J4	Suggestion automatique de KPI avec justification
Objectif	Vector analyse le dataset et propose 6 à 10 KPI pertinents, chacun avec son explication. Wow feature #2.
Tâches	• Endpoint POST /datasets/{id}/suggest-kpis • Prompt système dédié : « Tu es un expert analyse de données. Voici le schéma et le profil du dataset. Propose 6 à 10 KPI métier pertinents. Pour chaque KPI, donne son nom, sa formule (en utilisant les fonctions disponibles), son unité, et surtout son <i>why</i> : pourquoi cette KPI est utile pour cette activité. » • Schéma de sortie JSON strict via response_format=json_schema : {kpis: [{name, formula, unit, why}]} • Validation Pydantic du JSON retourné • Modale frontend qui affiche les suggestions avec : checkbox de sélection, nom, formule, badge unité, et un tooltip ou bloc qui affiche le « why » • Bouton « Tout sélectionner » et « Tout désélectionner » • Bouton « Importer les sélectionnés » qui crée les KPI en base • L'utilisateur peut éditer une suggestion avant import (corrections de formule)
Outils	OpenAI structured outputs, Pydantic, Radix UI
À maîtriser	JSON Schema strict, prompt engineering avec contraintes de format, UX de validation/import
Livrable	Vector suggère des KPI cohérents et explique pourquoi chaque KPI est pertinent. L'utilisateur trie et importe.
★ WOW	KPI avec justification — chaque KPI proposé vient avec un « why » qui transforme l'outil en conseil métier.

S3 J5	Polish KPI et Démo 3
Objectif	Robustesse, gestion des cas limites, tests end-to-end, enregistrement de la démo 3.
Tâches	- Gestion des cas limites : formule référençant une colonne supprimée du dataset, valeur NaN, division par zéro - Tests d'intégration end-to-end : upload → suggest → édition → save → compute → reorder → delete - États de chargement (skeletons) sur la page KPI pendant les appels lents - Messages d'erreur explicites en cas de problème : « Formule invalide : la colonne 'foo' n'existe pas dans ce dataset » - Animation discrète quand une KPI est recalculée (flash de la valeur) Enregistrer la Démo 3 Loom (3-5 minutes) : montrer suggestion auto, édition, recalcul, réordonner
Outils	Pytest, Playwright (optionnel pour E2E)
À maîtriser	Robustesse, gestion d'erreur UX, animations légères
Livrable	Démo 3 enregistrée. Système KPI fonctionnel et impressionnant.

Semaine 4 — Génération du dashboard HTML

Objectif de la semaine

Construire la fonction phare de Vector : génération automatique d'un dashboard HTML interactif avec cartes KPI, graphiques, sections d'analyse et section Recommandations toujours présente. Wow feature #3 : streaming du dashboard par section (apparition progressive).

Démo vidéo de fin de semaine

Démo 4 : sur un dataset chargé, je clique « Dashboard KPI général », je vois le dashboard se construire section par section sous mes yeux (titre → KPI cards → graphiques → analyse → recommandations), avec des graphiques ApexCharts interactifs et une section Recommandations visuelle en bas.

S4 J1	Schéma JSON du dashboard
Objectif	Définir un schéma JSON Pydantic complet qui sera le contrat entre le LLM et le moteur de rendu. Critical : tout le rendu HTML sera dérivé de ce JSON, donc il doit être complet et bien pensé.
Tâches	- Définir les classes Pydantic dans backend/dashboards/schemas.py - Schéma racine : Dashboard {title, subtitle, period, kpi_cards, charts, sections, recommendations} - KPICard : {name, value, unit, target?, evolution?, format} - Chart : {type (line/bar/donut/area/table), title, description?, config (ApexCharts), data} - Section : {title, content (Markdown), severity (info/warning/success)} - Recommendation : {priority (high/medium/low), action, impact, confidence?} - Documentation du schéma dans docs/dashboard_schema.md - Exemple d'un JSON dashboard complet pour test
Outils	Pydantic v2
À maîtriser	Schéma JSON, contrat d'interface LLM ↔ rendu, séparation données/présentation
Livrable	Schéma Pydantic complet, documenté, avec un exemple JSON valide.

S4 J2	Pipeline de génération côté backend avec streaming par section
Objectif	Construire le pipeline complet : profil → LLM (JSON) → calculs pandas → JSON final enrichi. Et stream chaque section pour effet « waouh » à l'écran.
Tâches	• Module backend/dashboards/generator.py • Fonction async generate_dashboard(dataset_id, intent) qui orchestre • Étape 1 : récupérer le profil du dataset (déjà calculé) • Étape 2 : appeler GPT-4o avec structured outputs pour produire le JSON dashboard • Étape 3 : pour chaque kpi_card du JSON, exécuter la formule via le formula_parser, remplacer la valeur • Étape 4 : pour chaque chart, calculer les données réelles avec pandas • Étape 5 : enrichir le JSON final avec metadata (généralisé, sur quel dataset) • Étape 6 : sauvegarder dans la table dashboards • ★ Streaming par section : au lieu d'attendre la fin, envoyer via WebSocket des événements {type: 'section_ready', section: 'kpi_cards', data: [...]} au fur et à mesure que chaque section est prête • Endpoint WebSocket /ws/datasets/{id}/dashboard
Outils	OpenAI, pandas, WebSocket, asyncio
À maîtriser	Pipeline orchestré, streaming d'événements structurés (pas juste de tokens), enrichissement de JSON
Livrable	Le backend produit un dashboard JSON enrichi en streaming par section.
★ WOW	Streaming par section — le frontend reçoit les morceaux du dashboard à mesure qu'ils sont prêts, créant l'effet « waouh ».

S4 J3	Template Jinja2 et rendu HTML autonome
Objectif	Convertir le JSON dashboard en HTML stylé, autonome (pas de dépendances externes), prêt à être affiché dans un iframe ou exporté en PDF.
Tâches	• Template backend/dashboards/templates/dashboard.html.j2 • Structure : <header> avec titre et période, <section class="kpi-grid"> avec les cartes, <section class="charts">, <section class="analysis">, <section class="recommendations"> • Tout le CSS est inline dans une balise <style> dans le HTML — pas de fichier externe (essentiel pour iframe srcDoc et export PDF) • Couleurs Vector : vert profond pour accents, blanc pour fond, noir pour texte • Section Recommandations : style visuel marqué (encart coloré, icônes par priorité) • Pour les graphiques : div placeholder avec data-attribute, ApexCharts les hydratera côté frontend • CSS print-friendly avec @media print : sauts de page propres, pas de hover • Service dashboard_render.py avec fonction render(dashboard_json) -> str
Outils	Jinja2, CSS, ApexCharts (placeholders)
À maîtriser	Templating, CSS inline, CSS print, autoescape Jinja2 (sécurité XSS)
Livrable	Le backend rend un HTML autonome propre, stylé, prêt à afficher ou exporter en PDF.

S4 J4	Viewer frontend avec ApexCharts et animations
Objectif	Afficher le dashboard côté frontend dans un iframe, avec hydratation des graphiques ApexCharts, et animations à l'apparition de chaque section.
Tâches	• Page <code>app/datasets/[id]/dashboards/[dashboardId]/page.tsx</code> • Composant <code>DashboardViewer</code> qui reçoit le JSON ou le HTML • Affichage du HTML dans un <code><iframe sandbox="allow-scripts" srcDoc={html}></code> • Quand le streaming envoie un événement <code>section_ready</code>, ajouter la section au DOM avec une animation fade-in (CSS <code>@keyframes</code>) • Pour les graphiques : iframe contient des placeholders <code><div data-chart="..." data-config="..."></code>, un script à la fin du iframe instancie ApexCharts dessus • Boutons en haut du dashboard : « Régénérer », « Télécharger PDF » (placeholder pour S5), « Partager le lien » • Indicateur de chargement pendant que les sections arrivent • Sauvegarder l'historique des dashboards générés (liste sur la page parent)
Outils	Next.js iframe <code>srcDoc</code> , ApexCharts, framer-motion (optionnel)
À maîtriser	iframe isolation, hydratation de scripts dans iframe, animations CSS
Livrable	Dashboard joliment affiché avec graphiques interactifs et animations à l'apparition.

S4 J5	Action « Dashboard KPI général » + Démo 4
Objectif	Bouton action rapide n°1 fonctionnel bout-en-bout. La section Recommandations est toujours présente. Tests sur 3 datasets, démo 4.
Tâches	• Sur la page Vector, la première carte d'action « Génère un dashboard KPI général » devient cliquable • Au clic : crée une nouvelle conversation, déclenche la génération, navigue vers la page dashboard, observe le streaming par section • Vérifier que la section recommandations est toujours présente dans le JSON LLM (instruction forte dans le prompt) • Section Recommandations stylée avec couleurs par priorité (rouge=high, ambre=medium, gris=low) • Tests sur 3 datasets : ventes, marketing, RH — vérifier que les KPI proposés sont cohérents à chaque fois • Enregistrer la Démo 4 Loom (3-5 minutes) : montrer le streaming par section, les graphiques, les recommandations
Outils	tout
À maîtriser	Intégration end-to-end, prompts robustes
Livrable	Démo 4 enregistrée. Première action rapide complète et impressionnante.

Semaine 5 — Analyses spécialisées + RAG

Objectif de la semaine

La semaine la plus chargée du plan, et la plus enrichissante. On construit les 4 autres actions rapides (Marketing, Commercial, Clients RFM, Tendances) **et** le pipeline RAG complet (ingestion + retrieval + injection). Deux wow features : Citation System et Data Quality Warnings dans les dashboards.

Démo vidéo de fin de semaine

Démo 5 : je teste les 5 actions sur un dataset, chacune produit un dashboard différent. Je télécharge un PDF. J'uploade un PDF de stratégie. Quand je relance une analyse, Vector cite explicitement les passages du PDF qu'il a utilisés pour enrichir ses recommandations.

S5 J1	Pipeline RAG — ingestion (parser, chunker, embedder)
Objectif	Construire le pipeline d'ingestion des documents : parsing, chunking, embedding, stockage dans pgvector. Asynchrone via Celery pour ne pas bloquer l'API.
Tâches	- Modèles SQLAlchemy Document (id, user_id, name, file_path, file_size, status, created_at) et Chunk (id, document_id, content, embedding Vector(1536), token_count, chunk_index) - Module backend/rag/parser.py : parse_pdf(path) avec pypdf, parse_docx(path) avec python-docx, parse_txt(path), parse_md(path) - Module backend/rag/chunker.py : découpe en chunks de ~500 tokens (mesurés via tiktoken) avec 50 tokens de chevauchement, en respectant les frontières de phrases - Module backend/rag/embedder.py : appelle text-embedding-3-small par batch de 100 chunks pour économiser les appels API - Endpoint POST /documents : upload (max 10 Mo), validation MIME, sauvegarde, déclenche la tâche Celery - Tâche Celery embed_document_task : parse + chunk + embed + store. Met à jour le statut (« parsing », « chunking », « embedding », « ready », « error ») - Endpoint GET /documents et GET /documents/{id}
Outils	pypdf, python-docx, tiktoken, openai embeddings, Celery, Redis, pgvector
À maîtriser	Chunking sémantique, embeddings batched, tâches asynchrones Celery, pgvector
Livrable	J'uploade un PDF de 20 pages, il est parsé, chunké en ~40 chunks, vectorisé, et indexé en quelques secondes.

S5 J2	Pipeline RAG — retrieval, injection, Citation System
Objectif	Recherche vectorielle, injection des chunks dans le prompt Vector, et wow feature : Vector cite les sources qu'il a utilisées.
Tâches	- Module backend/rag/retriever.py : fonction retrieve(query, user_id, top_k=5) -> List[Chunk] qui embed la query et fait un SQL avec opérateur cosinus <=> - Filtre seuil : ne garde que les chunks avec distance cosinus ≤ 0.7 - Scoping par utilisateur : on ne retrouve que les chunks des documents de l'utilisateur courant - Module backend/rag/injector.py : formate les chunks récupérés pour le prompt, avec marqueurs [Source 1: nom_doc.pdf, page X] - Intégration dans l'analyzer Vector : avant chaque appel au LLM, retrieve les chunks pertinents et les injecte ★ Citation System : le prompt système demande à Vector d'utiliser des références numérotées [1], [2], [3] quand il s'appuie sur une source. La réponse contient ces marqueurs. - Frontend : parser les marqueurs [N] dans la réponse, les remplacer par un lien qui ouvre un popover avec le contenu de la source - UI : à côté de chaque message Vector, badge « 3 sources » qui liste les sources utilisées (cliquable pour les voir)
Outils	pgvector, OpenAI, Pydantic, Radix Popover
À maîtriser	Recherche sémantique top-k, scoping multi-tenant, citation marker parsing, popover UX
Livrable	Vector cite explicitement les sources qu'il a utilisées. L'utilisateur peut cliquer et voir le passage exact.
★ WOW	Citation System — Vector ne dit plus « selon nos données », mais « selon la section 4.2 du plan stratégique 2026 [1] ».

S5 J3	Actions « Marketing » et « Commercial »
Objectif	Construire les actions rapides n°2 et n°3 avec leurs prompts dédiés et leurs variantes graphiques.
Tâches	• Action 2 — Analyse Marketing : prompt focalisé canaux d'acquisition (SEO, SEA, social), CAC, ROI, taux de conversion, funnel • Heuristique détection colonnes marketing : noms contenant « channel », « source », « campaign », « click », « impression » • Endpoint POST /datasets/{id}/dashboard?type=marketing • Action 3 — Analyse Commerciale : prompt focalisé CA, panier moyen, top produits, performance par vendeur, taux de transformation • Heuristique détection colonnes commerciales : « product », « salesperson », « amount », « date », « revenue » • Endpoint POST /datasets/{id}/dashboard?type=sales • Variantes graphiques : barres pour produits, courbes pour temporel, donut pour répartition canaux • Tests sur dataset adapté à chaque cas
Outils	OpenAI, pandas, ApexCharts
À maîtriser	Spécialisation par domaine, prompts métier, détection automatique de colonnes
Livrable	Actions 2 et 3 fonctionnelles, dashboards spécialisés produits.

S5 J4	Actions « Clients RFM », « Tendances », PDF export, Quality Warnings
Objectif	Terminer les 2 dernières actions, ajouter l'export PDF côté frontend, et intégrer la wow feature Data Quality Warnings.
Tâches	- Action 4 — Identification clients/segments : calcul RFM en pandas (R, F, M sur quantiles 1-5), classification 5 segments (Champions, Loyaux, Potentiels, À risque, Perdus) - Top-20 clients par CA - Endpoint POST /datasets/{id}/dashboard?type=customers - Action 5 — Tendances et anomalies : comparaison période actuelle vs précédente (auto-détection du grain), z-score et IQR pour anomalies - Endpoint POST /datasets/{id}/dashboard?type=trends Export PDF côté frontend : intégration html2pdf.js sur le bouton « Télécharger PDF ». Réglages : marges, sauts de page CSS, qualité images ★ Data Quality Warnings : sur les KPI calculés à partir d'une colonne avec >15% de NA, afficher un drapeau ■ avec tooltip « Confiance limitée : 18% de valeurs manquantes sur la colonne client_id » - Page app/documents/page.tsx complète : upload, liste, statut d'ingestion, suppression
Outils	pandas, scipy, html2pdf.js, Radix Tooltip
À maîtriser	Segmentation RFM, statistiques d'anomalie, conversion HTML→PDF dans le navigateur, UX de confiance
Livrable	5 actions complètes. PDF téléchargeable. Drapeaux qualité sur les KPI peu fiables. Page Documents opérationnelle.
★ WOW	Data Quality Warnings — l'utilisateur sait quels KPI sont solides et lesquels sont à interpréter avec précaution.

S5 J5	Intégration A+B (combo magique) + Démo 5
Objectif	Le moment de vérité : Vector utilise à la fois le dataset et les documents pertinents pour produire des recommandations enrichies.
Tâches	• Modifier le pipeline de dashboard : avant la génération du JSON, retrieve les chunks documents pertinents à partir de l'intent (« Analyse marketing » → chercher « strategy marketing 2026 » dans les docs) • Injecter ces chunks dans le prompt système du LLM avec instruction explicite de les utiliser pour enrichir les recommandations • Les citations apparaissent maintenant dans la section Recommandations du dashboard • Test du combo : dataset de ventes + PDF « Plan stratégique 2026 » → analyse → recommandations qui font référence au plan • Polish UI : indication subtile du nombre de documents utilisés en contexte (« 12 sources consultées ») • Enregistrer la Démo 5 Loom (5 minutes) : la plus impressionnante, montrer combo data + RAG + citations + quality warnings
Outils	tout
À maîtriser	Orchestration multi-pipeline, prompt enrichi multi-source, démo scénarisée
Livrable	Démo 5 enregistrée. Vector combine données chiffrées et contexte textuel pour des recommandations vraiment éclairées.

Semaine 6 — Wow features, polish, rapport

Objectif de la semaine

La semaine où on transforme un bon projet en projet impressionnant. Trois wow features finales (Comparaison, Conversational Refinement, Smart Caching), robustesse, polish UX, audit Lighthouse, rédaction du rapport et démo finale scénarisée.

Démo vidéo de fin de semaine

Démo finale : 8 à 10 minutes scénarisées qui montrent tout le flow : login → upload dataset → upload PDF stratégie → suggestion KPI avec why → dashboard avec streaming → comparaison de périodes → refinement conversationnel → citations RAG → export PDF. Le tout fluide, animé, professionnel.

S6 J1	Mode Comparaison + Conversational Refinement
Objectif	Deux wow features qui changent la donne : comparer deux périodes côte à côte, et modifier un dashboard existant via conversation.
Tâches	★ Mode Comparaison : endpoint POST /datasets/{id}/dashboard?type=...&compare=true._a=..._b=... - Le dashboard généré a un layout 2 colonnes : Période A à gauche, Période B à droite - Chaque KPI card affiche les 2 valeurs avec le delta (% évolution, flèche colorée) - Chaque graphique a 2 séries (Période A et Période B) - UI : sélecteur de période avec date pickers ou presets (Mois dernier vs Mois précédent, Q1 vs Q4, etc.) ★ Conversational Refinement : sur la page d'un dashboard généré, un chat input « Affine ce dashboard... » - Au submit : envoie le JSON du dashboard actuel + la demande de modification au LLM, qui retourne un nouveau JSON modifié - Exemples supportés : « exclus la région Nord », « ajoute un KPI panier moyen », « change la période en Q3 », « focus sur les produits de la catégorie X » - Animation à la transition : ancien dashboard fade out, nouveau fade in - L'historique des refinements est sauvegardé (chaque version est gardée)
Outils	pandas, OpenAI, Next.js, framer-motion
À maîtriser	State management, conversational refinement, diff visuel entre versions
Livrable	Mode comparaison opérationnel. L'utilisateur peut affiner un dashboard par phrase courte.
★ WOW	Comparaison + Refinement — deux features qui démultiplient l'utilité de chaque dashboard généré.

S6 J2	Smart Caching et performance
Objectif	Optimiser la performance et le coût : ne pas recalculer ce qui peut être réutilisé. Wow feature #8.
Tâches	★ Smart Caching : cache Redis du profil statistique de chaque dataset (clé : <code>profile:{dataset_id}:{file_hash}</code>) - Cache des embeddings : si un document est uploadé deux fois (même hash), réutiliser les embeddings - Cache des dashboards générés : si l'intent est identique et le dataset n'a pas changé, proposer la version cachée (avec bouton « Régénérer ») - Cache du résultat des recherches RAG par query (TTL 1h) - Métriques de cache hit/miss exposées sur un endpoint <code>/metrics</code> - Pagination des listes (datasets, documents, dashboards) avec <code>limit/offset</code> - Index DB optimisés (vérifier les <code>EXPLAIN ANALYZE</code> des requêtes lentes) - Compression Brotli des réponses API - Lazy loading des images dans le frontend
Outils	Redis, SQLAlchemy, Next.js Image, EXPLAIN ANALYZE
À maîtriser	Caching strategies, cache invalidation, indexation, performance web
Livrable	App réactive même avec plein de données. Coût LLM divisé d'au moins 30% sur les cas répétitifs.
★ WOW	Smart Caching — Vector réutilise intelligemment ses propres calculs précédents, ce qui le rend ultra-rapide en utilisation réelle.

S6 J3	Robustesse, accessibilité, audit Lighthouse
Objectif	Cas limites, erreurs gracieuses, accessibilité, score Lighthouse > 90.
Tâches	• Cas limites identifiés et traités : dataset vide, dataset 1 ligne, colonnes mal nommées, dates au mauvais format, valeurs négatives partout, doublons à 100% • Cas limite LLM : retry automatique si JSON cassé (max 2 retries avec prompt correctif), fallback message clair si tout échoue • Cas limite réseau : timeout OpenAI configuré à 60s, message UX explicite « Réessaie dans un moment » • Messages d'erreur uniformes : pas de stack trace, langage métier • Accessibilité : navigation au clavier (tab, esc, enter), attributs ARIA, focus visible, contrastes WCAG AA • Audit Lighthouse sur les principales pages : objectif >90 sur Performance, Accessibility, Best Practices, SEO • Tests E2E des 5 actions rapides (Playwright si possible, sinon manuels documentés) • Documentation des cas connus dans docs/KNOWN_ISSUES.md
Outils	Pytest, Playwright, Lighthouse, axe-core (a11y)
À maîtriser	Graceful degradation, accessibilité web, audit qualité
Livrable	App robuste, Lighthouse >90, accessible au clavier, gestion d'erreur exemplaire.

S6 J4	Rédaction du rapport principal
Objectif	Rédiger le corps du rapport pro (80% du contenu). Diagrammes, captures, métriques.
Tâches	• Plan détaillé du rapport (60-80 pages estimées) : Remerciements, Contexte entreprise, Présentation de la plateforme, Objectifs du stage, État de l'art (LLM, RAG, dashboards), Architecture choisie avec diagrammes, Choix techniques justifiés, Réalisation semaine par semaine, Résultats, Limites et perspectives, Conclusion, Annexes • 3-4 diagrammes d'architecture (draw.io ou Excalidraw) : global, flux d'une requête de dashboard, pipeline RAG, schéma de base de données • Captures d'écran soignées de chaque écran principal • Métriques mesurées : temps moyen de génération d'un dashboard (sur 100k lignes), coût OpenAI par dashboard (en \$), nombre moyen de tokens, taux de cache hit • Section « Choix techniques » qui justifie : Jinja2 plutôt que HTML LLM, pandas pour calculs, structured outputs, AST whitelisting, citation system • Section « Wow Features » avec une page dédiée par feature : problème résolu, implémentation, valeur ajoutée
Outils	Microsoft Word, Google Docs, ou LaTeX (Overleaf), draw.io, Excalidraw
À maîtriser	Rédaction technique professionnelle, justification de choix, structuration de rapport
Livrable	Rapport à 80% terminé (manque introduction, conclusion, relecture).

S6 J5	Démo finale scénarisée, finalisation rapport, livraison
Objectif	Enregistrer la démo finale impressionnante, finaliser le rapport, livrer le tout à l'encadrant.
Tâches	• Préparer un dataset de démonstration soigneusement choisi (ventes B2B 2024, 100k lignes, propres) • Préparer un document PDF de stratégie business pour le RAG (3-4 pages, fictif mais cohérent) • Écrire un script de démo détaillé (timing minute par minute, 8-10 minutes total) • Le script doit montrer : login, upload du dataset, score qualité, upload du PDF, suggestion KPI avec why, dashboard avec streaming, citations RAG, comparaison de périodes, refinement conversationnel, export PDF • Enregistrer la démo Loom finale en plusieurs prises si nécessaire • Rédiger introduction et conclusion du rapport • Relecture complète, correction des fautes, vérification de la pagination • Export PDF final du rapport, vérification de la mise en page • Envoi à l'encadrant : vidéo de démo + rapport PDF + lien GitHub du projet • Bilan personnel : ce qu'on a appris, ce qu'on changerait, fierté du livrable
Outils	Loom, Word/LaTeX, lecteur PDF
À maîtriser	Pitch, scénarisation, relecture critique, bilan de stage
Livrable	Démo finale Loom envoyée + Rapport PDF final livré. Projet officiellement terminé.

10. Risques et plan de mitigation

Risque	Probabilité	Mitigation
Format HTML imposé pas reçu à temps	Moyenne	Continuer avec format par défaut, adapter quand reçu (le template Jinja2 est facilement modifiable)
Quotas OpenAI atteints	Faible	Smart caching réduit la conso, utilisation de gpt-4o-mini pour validations, monitoring du budget
Performance dégradée sur datasets 100k lignes	Moyenne	Profiling pandas optimisé, agrégations avant graphique, échantillonnage automatique au-delà
Bug critique en dernière semaine	Moyenne	Tests E2E dès S5, suite de tests qui couvre les 5 actions, rollback Git possible
LLM retourne du JSON cassé	Faible avec structured outputs	Retry automatique avec prompt correctif (max 2 fois), fallback message UX
Retard sur la semaine 5 (charge dense)	Moyenne	Prioriser RAG (indispensable pour démo finale), reporter une action en S6 J1 au pire
Encadrant indisponible pour démos	Faible	Vidéo Loom asynchrone (avantage natif), pas besoin de RDV synchrone

11. Conseils pratiques au quotidien

11.1 Routine quotidienne idéale

- **9h00** — Ouvrir le plan, lire la fiche du jour. 5 minutes.
- **9h05** — Reprendre où on en était la veille (Git status, dernier commit, journal de bord)
- **9h15-12h30** — Code focalisé. Petits commits fréquents avec messages descriptifs.
- **14h-17h30** — Code, tests, captures d'écran si feature impressionnante
- **17h30** — Bilan du jour : ce qui marche, ce qui coince, journal de bord
- **17h45** — Push Git, planifier la 1ère tâche du lendemain

11.2 Le journal de bord

Crée un fichier JOURNAL.md dans le repo dès J1. À la fin de chaque journée, écris 5 à 10 lignes :

- Ce que j'ai fait aujourd'hui (tâches accomplies)
- Ce qui m'a bloqué et comment je l'ai résolu
- Ce que j'ai appris de nouveau
- Ce que je fais demain en priorité

Ces notes seront la **matière première du rapport final**. Sans elles, en S6 tu auras oublié 70% de ce que tu as appris.

11.3 Les démos Loom hebdomadaires

- Toujours faire la démo le **vendredi en fin de journée**, sur la version stable du jour
- Préparer un mini-script (5 lignes) de ce que tu veux montrer
- Tester la prise une fois sans enregistrer, pour caler le timing
- Enregistrer en plusieurs prises si besoin, garder la meilleure
- Envoyer le lien Loom à l'encadrant **dès que c'est fait**, avec 3 lignes de résumé
- Sauvegarder les vidéos en local aussi (au cas où Loom évolue)

11.4 Gestion du code Git

- Commits petits et fréquents : un commit = une idée
- Messages de commit en anglais, style « feat: », « fix: », « refactor: », « docs: »
- Branche main = stable, démos enregistrées dessus
- Branches feature/xxx pour les chantiers longs (citations, mode comparaison, etc.)
- Push tous les soirs au minimum (backup distant)
- Ne jamais commit le .env (clé OpenAI = catastrophe sur GitHub)

11.5 Quand on est bloqué

- Si bloqué 30 minutes sur un même problème : **stop**, écris le problème par écrit
- Reformuler en 3 phrases : ce que je veux faire, ce qui se passe, ce que j'ai essayé
- Souvent, écrire suffit à débloquent (effet « canard en plastique »)
- Sinon : demander à Claude (moi) avec le problème écrit + extrait de code + message d'erreur
- Sinon : demander à l'encadrant via message asynchrone, pas urgent

11.6 Préserver la motivation sur 6 semaines

- Chaque vendredi, regarder la démo de la semaine précédente : voir le chemin parcouru
- Célébrer les petites victoires (un premier embedding qui marche, un dashboard joliment rendu)
- Ne pas viser la perfection sur chaque tâche : **fini vaut mieux que parfait**
- Si une journée est foirée : passer à autre chose, ne pas se flageller
- Si une semaine est en retard : on prend sur la S6 (qui a du polish réutilisable comme buffer)
- Bon stage = bon dossier futur. Chaque ligne de code construit ton portfolio.

✓ Le mot de la fin

Ce plan est ambitieux mais tenu, et il fera de Vector un projet remarquable. À chaque instant tu peux dire « stop, explique-moi » et on n'avance pas tant que tu n'es pas à l'aise. La qualité prime sur la vitesse, et la compréhension prime sur la quantité de code. Bon courage — on se revoit au J1.